Desenvolvimento Web e Apps

Introdução ao **CSS**

Prof. Diógines Goldoni (Dio)

Introdução ao CSS

CSS

Temos 3 formas principais de adicionar CSS ao HTML

- **Inline**: Quando os estilos são adicionados por um atributo;
- **Internal**: Quando o CSS é adicionado na tag head;
- **External**: Quando o CSS é importado no HTML, vindo de um arquivo externo.
    - Esta é a forma ideal

Anatomia do CSS

Para aplicarmos estilo CSS a um elemento:

1. Selecionamos o elemento que desejamos aplicar o estilo, via tag, id ou classe

2. Depois adicionamos as propriedades (ex.: font-style) e valores (ex.: italic)

A sintaxe é: nome_da_ propriedade : valor;

Exemplo:

```
font-style: italic;

color: red;
```

Inline CSS

O CSS inline pode ser adicionado diretamente em um elemento, como atributo!

```
<!--CSS Inline-->
<h2 style="color: blue;">Mudar cor com o Inline CSS</h1>
<h2>Outro título, voltando a cor original</h1>

<!-- Múltiplas regras inline CSS-->
<h2 style="color: purple; font-size: 45px; border-bottom: 1px solid red;">Muda cor, tamanho e borda</h1>
```

Prática

Internal CSS

O CSS interno é melhor que o inline, onde *todos* os estilos vão na tag head!

Tais regras de estilo precisarão ser envoltas pela tag `<style> </style>`

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <title>CSS</title>
    <!--CSS Interno-->
    <style>
        p {
            color: gold;
            font-size: 22px;
        }
    </style>
</head>
<body>
    <p>Eu fui estilizado via CSS interno</p>
    <p>Eu também</p>
</body>
</html>
```

Prática

External CSS

Para esta técnica, criamos um arquivo .css, normalmente dentro de uma pasta chamada css

Precisamos importar este arquivo através da *self-closing* tag `<link>` dentro da tag <head></head>

```
<head>
    <title>CSS</title>
    <!--CSS Externo-->
    <link rel="stylesheet" href="css/styles.css" />
</head>
```

Dentro do arquivo styles.css:

```
body {
    background-color: #fff;
}
h3 {
    color: darkgreen;
}
```

Prática

Ordem de precedência do CSS

O CSS é lido pelo HTML seguindo uma ordem!

Se tivermos estilos misturados, *inline, internal, external*, a ordem será:

inline > internal = external > padrão do navegador

Múltiplos arquivos de CSS

É possível termos mais de uma folha de estilo CSS no nosso projeto!

Precisamos apenas importar todas elas na tag <head> </head>

- A ordem de importância é inversa, logo, o arquivo *n + 1* importado terá precedência sobre o *n*
- Utilizar múltiplos arquivos CSS é boa prática, pois facilita a divisão do CSS por páginas, por exemplo

Prática

Classes e IDs

Classes e IDs são atributos de tags HTML, e facilitam a sua manipulação por CSS e Javascript

- IDs serão utilizados para estilizar elementos únicos, como o título da página, por exemplo
- Classes servem para atribuirmos um ou mais estilos para um conjunto de elementos distintos

Classes

Adicionar o atributo `class="nome_da_classe"` é o suficiente para manipularmos ela via CSS

- Ex.: Se quisermos setar o estilo de todos os parágrafos na página:

No HTML:

```html
<p class="paragrafo">Algum texto 1</p>
<p class="paragrafo">Algum texto 2</p>
<p class="paragrafo">Algum texto 3</p>
```

No CSS:

```css
/* O seletor CSS para classes é denotado por um . antes do nome da classe */
.paragrafo {
    background-color: #000;
    color: aqua;
    padding: 5px;
}
```

Prática

IDs

IDs também são atributos do HTML, no formato `id="nome_do_id"`

- Ex.: Se quisermos setar o estilo de um título em específico na página:

No HTML:

```html
<h1 id="titulo-exclusivo">Este é um título com ID</h1>
```

No CSS:

```css
/* O seletor CSS para IDs é denotado por um # antes do nome do ID */
#titulo-exclusivo {
  background-color: purple;
  color: yellow;
  padding: 10px;
  font-size: 40px;
}
```

Prática

Ordem dos seletores

E se uma tag tiver ID e Classe? A ordem será:

id > class > seletor de tag

Logo, o ID irá ter maior precedência.

IMPORTANTE:

A ordem de precedência só se aplica caso haja algum conflito, por exemplo, uma tag p que tenha o ID e classe, onde ambas setam valores diferentes para seu tamanho de fonte e cor. Nesse caso, o tamanho e cor que tomará precedência serão os definidos no CSS via ID!

Prática

Cores no CSS

No CSS, podemos acessar cores usando diferentes formas

- Usando o nome literal das cores, como ***red*** ou ***blue*** (não muito utilizado)
- RGB: Configurando a tonalidade de ***red, green*** e ***blue***
- Hexadecimal: Um código de 6 a 8 dígitos que nos permite definir mais de 16 milhões de cores
    - Forma mais utilizada!
- HSL: ***hue, saturation*** e ***lightness***

Hexadecimal

Abordagem mais utilizada!

- Basicamente, precisamos inserir 6 dígitos precedidos por # como valor para um atributo de cor
- Os dois 2 dígitos representam o tom de vermelho, depois 2 para verde e 2 para azul
- Os valores vão de A a F e 0 a 9
- O número 0 representa preto, e o F o branco, ou seja #000000 é preto e #FFFFFF branco
  - Caso um valor seja repetido 6 vezes como acima, podemos escrever com apenas 3 caracteres #000 ou #FFF
- Para lidarmos com transparência, adicionamos mais 2 dígitos no hexadecimal, onde:
  - Quanto mais próximo de FF os 2 últimos dígitos forem, mais opaca é a transparência
  - Quanto mais próximo de 00 os 2 últimos dígitos forem, mais transparente o elemento se torna
- Podemos atingir o mesmo resultado usando o atributo CSS opacity, que vai de 0.0 a 1.0

Prática

Background images

Podemos inserir imagens como fundo dos elementos HTML

- Para tal, utilizamos o atributo CSS `background-image: url("pasta/imagem.jpg");`

Prática

Centralizando imagem de fundo

É comum termos uma imagem maior que o elemento que a comporta

Para melhorarmos isto, podemos centralizar a imagem, utilizando 2 regras

- Setamos o valor do atributo `background-position: center;`
- E o valor do atributo `background-size: cover;`

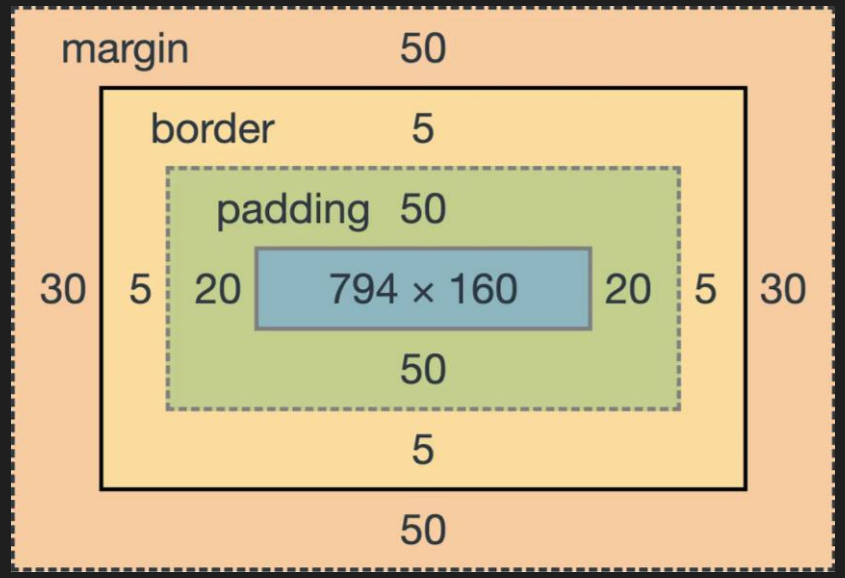
Prática

Box Model

O box model é uma entidade criada em todo elemento HTML

Consiste em 4 partes: height/width, padding, border e marging

- Todas estas partes do box model podem ser alteradas via CSS
- Browsers setam valores padrão de padding e marging para diversos elementos
  - Podemos (e devemos!) "zerar" estes valores para que nossos layouts fiquem conforme o layout dos desginers



Altura e largura: height e width

A altura e largura são a base do box model, e permitem que mudemos o tamanho de elementos

Como vimos na aula passada, *block elements* ocupam por padrão 100% da largura da tela!

- Mais adiante, veremos com alterar este comportamento, com o uso de divs e CSS

Prática

Padding

O padding é o espaço entre o conteúdo de uma tag e sua borda (borda fictícia, chamada de box)

O padding também é conhecido como **espaçamento interno**

- Utilizado para criar uma distância entre o conteúdo e as extremidades do elemento.

Prática

Padding em coordenadas específicas

Podemos adicionar o padding a lados específicos do nosso elemento usando:

```
padding-top: 10px;

padding-left: 20px;

padding-bottom: 10px;

padding-right: 15px;
```

Ou via *shorthand property* p/ aplicar valores distintos com uma só linha, em sentido horário:

```
padding: 10px 20px 10px 15px;
```

Diversas regras CSS aceitam shorthand properties, como margin e border

Prática

Importante: padding e width

Cuidado, pois o valor do padding é adicionado a largura do elemento!

- Um elemento com `width: 200px e padding: 25px` tem um tamanho horizontal total de 250px!
- Até podemos resolver isso diminuindo a largura do elemento para 150px, mas não seria o ideal...
- Esse problema pode ser resolvido com a regra `box-sizing: border-box`
  - ✓ Isto irá forçar com que o elemento respeite o tamanho que está no width!

Prática

Border

A borda de um elemento fica entre padding (espaçamento interno) e margin (externo)

- Este propriedade é normalmente estilizada com propósito decorativo
- Borders podem ser setadas em lados específicos
  - ✓ border-top, border-left, border-bottom, border-right
- A utilização de uma `div` com `border-bottom: 1px;` é preferível, ao invés de um `<hr />`
- As propriedades principais de `border` são `size`, `type` e `color`

Podemos setar as 3 principais propriedades usando *shorthand*:

```
.border {
    border: 5px solid red;
}
```

Prática

Bordas arredondadas: border-radius

Com a propriedade `border-radius` podemos arredondar os cantos de um elemento

- Podemos arredondar elementos como divs, botões, caixas de texto, etc.

```
border-radius: 5px;
```

Prática

Margin

A propriedade `margin` é responsável pelo espaçamento *entre* o elemento em questão e os outros

- Podemos aplicar o recurso como aplicamos o **`padding`**
- Podemos aplicar as propriedades **`margin`** a lados específicos, e também usar *shorthand*

Prática

Juntando tudo!

É comum que apliquemos todos os elementos do box model em alguns elementos, ou seja:

- Definimos seu tamanho com `width` e `height`
- Adicionamos espaçamento interno com `padding`
- Decoramos o elemento com `border`
- Afastamos o elemento de outros com `margin`

Prática

text-align

Podemos alinhar texto em várias direções com `text-align`

- Por padrão, o texto é alinhado a esquerda
- Podemos alterar o valor para `center` ou `right`, ao invés do padrão que é `left`

text-align

Podemos alinhar texto em várias direções com `text-align`

- Por padrão, o texto é alinhado a esquerda
- Podemos alterar o valor para `center` ou `right`, ao invés do padrão que é `left`

text-decoration

Podemos adicionar ou remover efeitos de texto

- Podemos tornar o texto `bold`, `italic`, `line-through`, `underline`, e outras possibilidades
- A tag `<a> </a>` para gerar links, vêm com sublinhado por padrão. Podemos remover, se quisermos.

Prática

text-transform

Podemos alterar como o texto é exibido com `text-transform`

- Alguns valores possíveis são `uppercase` e `lowercase`
- Cuidado: É má prática escrever texto em letra maiúscula no HTML. Use o CSS para tal!

text-transform

Podemos alterar como o texto é exibido com `text-transform`

- Alguns valores possíveis são `uppercase` e `lowercase`
- Cuidado: É má prática escrever texto em letra maiúscula no HTML. Use o CSS para tal!

letter-spacing

Podemos alterar o espaçamento entre letras de um texto com `letter-spacing`

Prática

Fontes

Com o CSS, podemos alterar o tipo da fonte com a regra `font-family`

- Por padrão, temos as seguintes regras disponíveis são: Serif, Sans-serif, Monospace, Cursive e Fantasy
- Todos navegadores possuem uma vasta quantia de fontes que podemos utilizar por padrão
- Além disto, podemos adicionar fontes externas

Fontes

Com o CSS, podemos alterar o tipo da fonte com a regra `font-family`

- Por padrão, temos as seguintes regras disponíveis são: Serif, Sans-serif, Monospace, Cursive e Fantasy
- Todos navegadores possuem uma vasta quantia de fontes que podemos utilizar por padrão
- Além disto, podemos adicionar fontes externas

font-style

Podemos alterar o aspecto das letras com `font-style`

- Os valores são `normal`, `italic` e `oblique` (itálico com poucas diferenças)

Fontes

Com o CSS, podemos alterar o tipo da fonte com a regra `font-family`

- Por padrão, temos as seguintes regras disponíveis são: Serif, Sans-serif, Monospace, Cursive e Fantasy
- Todos navegadores possuem uma vasta quantia de fontes que podemos utilizar por padrão
- Além disto, podemos adicionar fontes externas

font-style

Podemos alterar o aspecto das letras com `font-style`

- Os valores são `normal`, `italic` e `oblique` (itálico com poucas diferenças)

font-weight e font-size

font-weight: Utilizado para deixar nossa fonte mais fina ou grossa, com valores entre 100 e 900

font-size: Utilizado para setarmos o tamanho da fonte, usando um valor em px, por exemplo

Prática

Display

Em HTML e CSS, há block elements e inline elements

- A tag `<div>` é um exemplo de block element
- A tag `<span>` é um exemplo de inline element
- Com a regra `display` podemos mudar esse comportamento!
- Os principais valores para display são `inline-block`, `block` e `none`
- Atenção: O `display: none;` fará com que o elemento fique invisível, porém ainda exista no HTML

Prática

Posicionando elementos

A regra `position` e seus valores são utilizados para posicionar o elemento na tela!

Os principais valores são: `static`, `relative`, `fixed`, `absolute` e `sticky`

- O valor padrão é `static`, e todo elemento é renderizado com esta posição se não for alterado

Posicionando elementos

A regra `position` e seus valores são utilizados para posicionar o elemento na tela!

Os principais valores são: `static`, `relative`, `fixed`, `absolute` e `sticky`

- O valor padrão é `static`, e todo elemento é renderizado com esta posição se não for alterado

position: static

Este valor não faz nada com o elemento, sendo o elemento padrão...

Outros valores de `position` são afetados pelas regras `top`, `left`, `right` e `bottom`, mas o `static` não.

Prática

position: relative

O elemento é posicionado em relação a sua posição normal, porém permite alterações via:

`top, right, bottom, left`

Prática

position: absolute

Remove o elemento do fluxo normal do HTML, fazendo com que ele não seja afetado pelos elementos que o rodeiam. Permitirá o posicionamento do lemento em qualquer local da tela.

- O elemento é posicionado com relação ao seu vizinho mais próximo com algum valor de `position` que não seja o padrão (`static`).
- Caso não encontre, o elemento será posicionado com relação ao HTML (canto superior esquerdo)

Prática

position: absolute dentro de um position: relative

Com um container com `position: relative` podemos controlar melhor a área de ação dos elementos com `position: absolute`

Prática

position: fixed

Com este tipo de posicionamento, podemos fixar um elemento na tela!

- Mesmo após o scroll na página, o elemento permanece na mesma posição
- O recurso é frequentemente utilizado para criar barras de navegação fixas, como menus de topo

Prática

position: sticky

Também faz com que o elemento fique fixo na tela...

Mas quando o estamos visualizando a posição original do elemento, ele se comporta como `relative`

- No caso, a posição original do elemento é onde ele foi originalmente inserido no HTML

Prática

z-index

Se temos dois ou mais elementos com as mesmas posições, ou se eles colidem na página, podemos escolher qual será exibido em cima com o `z-index: valor;`

- Quanto maior o valor atribuído ao **`z-index`**, mais acima dos outros ele estará

Prática

Perguntas?